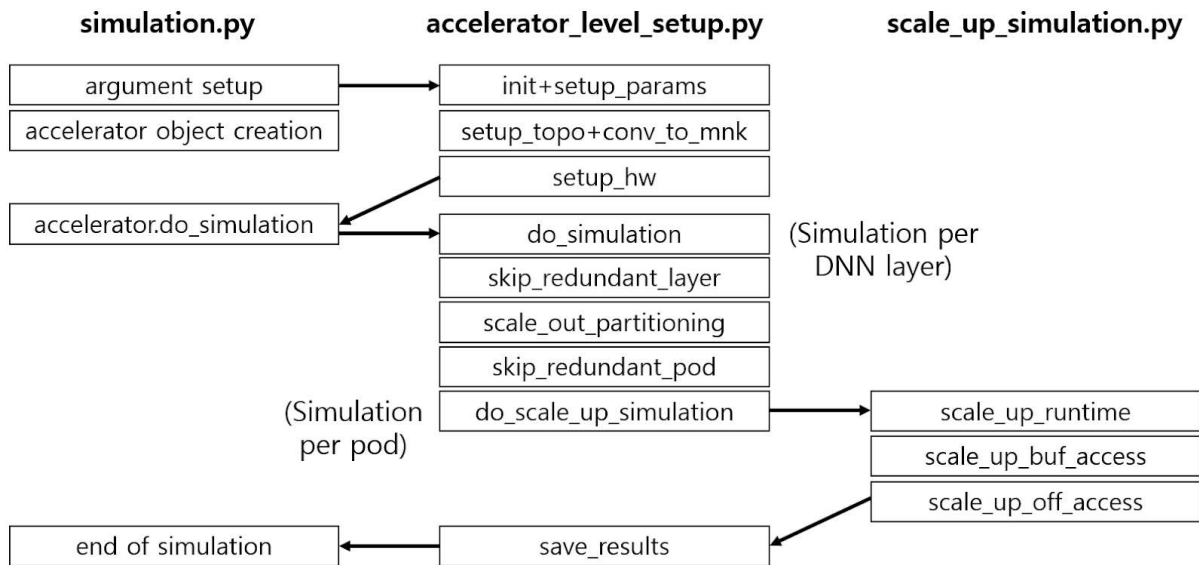


Modified Simulator Guide

1. Simulator Overview



< Simulator flow >

simulation.py

시뮬레이션 실행 스크립트로서, argument를 parsing하고 accelerator_level_setup object를 생성함. 이후, 전체 시뮬레이션을 시작함.

accelerator_level_setup.py

이 스크립트는 가속기 설정 및 시뮬레이션 실행을 담당하는 핵심 스크립트임. 시뮬레이션은 DNN layer마다, 그리고 pod 단위로 계층적으로 이루어짐.

먼저, workload를 구성하는 layer 단위로 시뮬레이션을 수행하며, 이 과정에서 이미 동일한 형태의 layer에 대해 시뮬레이션이 진행되었을 경우 skip하도록 함. 이후, dataflow를 고려해 scale-out DNN accelerator에서 각각의 pod에 할당할 matrix를 분할함. Matrix 분할 후, 서로 동일한 동작을 수행하게 되는 pod들에 대해, 매번 반복해 시뮬레이션 하지 않고 skip함. 동일한 연산을 수행하는 layer에 대해서도 매번 반복해 시뮬레이션하지 않고 skip함.

scale_up_simulation.py

Scale-out DNN accelerator의 각 pod에 대해, analytical model을 활용한 simulation을 수행함. Runtime, 각 data type (input, weight, output)별 on-chip 및 off-chip memory access 횟수를 계산함. 계산 결과를 accelerator_level_setup에게 반환함.

2. Input and Output Files

```
1  [NPU_others]
2  pod_dimension_row: 4
3  pod_dimension_col: 2
4  clock_frequency: 1050
5  bandwidth: 10000
6  latency: 1
7  dataflow: WS
8
9  [NPU_systolic]
10 row: 32
11 col: 32
12 input_buffer: 64
13 weight_buffer: 64
14 output_buffer: 64
```

< 예시 hardware configuration file >

pod_dimension_row, _col: Scale-out DNN accelerator에서, pod들의 구성을 나타냄.

clock_frequency: Accelerator chip의 MHz 단위의 clock frequency를 나타냄 (현재 시뮬레이션에서는 사용되지 않음).

bandwidth, latency: Off-chip memory bandwidth 및 latency를 나타냄. SCALE-Sim과 동일하게, 현재 시뮬레이션에서는 고려하지 않음.

dataflow: DNN accelerator의 dataflow를 나타냄 (IS, WS, OS).

row, col: Scale-out DNN accelerator에서, 각 pod 내의 systolic array dimension을 나타냄.

input_, weight_, output_buffer: Scale-out DNN accelerator에서, 각 pod 내의 buffer size를 KB 단위로 나타냄.

```
1  small, 32, 1, 1, 1, 256, 32, 1,
2  large, 64, 1, 1, 1, 256, 64, 1,
```

< 예시 topology file (convolution layer format) >

Workload에 대한 정보를 포함하는 topology file은 convolution layer format과 MNK format의 두 가지의 format으로 작성 가능하며, csv 파일 내에 각 줄마다 한 layer에 대한 정보를 담고 있음.

Convolution layer format은 input feature map (IFMAP)과 filter에 대한 dimension과 stride 정보를 포함함. 파일 내의 값들은 왼쪽부터 순서대로 Layer name, IFAMP Height, IFMAP Width, Filter Height, Filter Width, Channels, Num Filter, Strides를 의미함.

```

1  small,32,32,256,
2  large_input,64,32,256,
3  large_weight,32,64,256,
4  large_all,64,64,256,

```

< 예시 topology file (MNK format) >

GEMM operation에서 사용되는 M, N, K dimension으로 작성되어 있음. 이 때, input matrix와 weight matrix의 dimension은 각각 (M, K), (K, N)임. 이 dimension을 정리하면 아래와 같음.

M: Input matrix의 row size.

N: Weight matrix의 column size

K: Input matrix의 column size (= weight matrix의 row size)

Layer	Runtime	Input Buffer Access	Weight Buffer Access	Output Buffer Access	Input Off-chip Access	Weight Off-chip Access	Output Off-chip Access
0	318	8192	32768	1024	8192	32768	1024
1	318	16384	32768	2048	16384	32768	2048
2	636	16384	65536	2048	8192	65536	2048
3	636	32768	65536	4096	16384	65536	4096

< 예시 결과 파일 >

Layer 단위로 runtime과 on-chip buffer access, off-chip memory access에 대한 결과값을 도출함. Access count는 input, weight, output 각 data로 인해 발생한 access를 구분해서 계산함.

3. Running Simulation

```

$ python3 simulation.py -t <path_to_topology_file>
-c <path_to_config_file> -i <input type (gemm/conv)>

```

```

(Example)
$ python3 simulation.py -t $PWD/topology/microbench/microbenchmark.csv
-c $PWD/config/ex_mnk_os.cfg -i "gemm"

```

< 예시 실행 명령어 >

시뮬레이터 실행 명령어는 그림과 같음. 상단 그림의 명령어는 실행 명령어의 일반적인 구조를 나타냄. topology file, config file, topology file의 형태 (gemm or conv)를 입력해 실행함.

아래 그림의 명령어는 실제 실행 명령어 예시를 나타냄. “gemm” 형태로 작성된 “microbenchmark.csv”라는 topology file과, “ex_mnk_os.cfg”라는 하드웨어 configuration으로 실험을 진행함.

4. Running SCALE-Sim

Analytical model의 검증을 위해, SCALE-Sim을 실행할 수 있음. SCALE-Sim은 실험 설정 방법과 실행 명령어가 다름. 차이점 및 실행 방법을 간략히 기술함.

```
562 if __name__ == '__main__':
563
564     parser = argparse.ArgumentParser(description='')
565     parser.add_argument('topo', help='Set topology file')
566     args = parser.parse_args()
567
568     simname= 'Scaleout'
569     toponame= args.topo
570     plotname_runtime = 'RT.png'
571     plotname_DRAM = 'DRAM.png'
572     mnk_flag = True
573     dflow = 'ws'
574     topofile = './topologies/microbench/'+toponame+'.csv'
575     config_base = './configs/'
576
577     # (pod dimension : config file name)
578     grid_arr_dict = {'4x2': 'ex_mnk_ws'}
579     #####
```

< Scale_out_sim_ws.py 스크립트 예시 >

SCALE-Sim 실행 스크립트는 dataflow에 따라, Scale_out_sim_is, ws, os 3가지 스크립트로 분리되어 있음. 위 사진은 ws 스크립트의 예시임. 유저가 설정할 필드는 dflow, topofile, grid_arr_dict 세 가지임.

dflow: ws, is, os 세 가지 dataflow 중 하나를 입력. config 파일의 dataflow가 자동으로 입력되지 않음.

topofile: topology file의 경로를 의미하며, toponame은 실행 명령어에서 전달하는 argument임. toponame 이전 상위 경로 수정이 필요할 경우 스크립트에서 직접 수정이 필요함.

grid_arr_dict: Scale-out DNN accelerator 구조 및 config 파일 이름을 입력함. 4x2는 4개 row, 2개 column으로 구성된 구조를 뜻함. Config file은 개별 pod 내의 설정에 대한 정보를 담고 있음.

```
$ python3 Scale_out_sim_ws.py microbenchmark
```

< Scale_out_sim_ws.py 스크립트에 대한 실행 명령어 예시 >

위 명령어는 “microbenchmark”라는 topology에 대한 시뮬레이션을 실행하는 명령어임. Hardware configuration은 스크립트 내에 작성된 내용을 따라 설정됨.

시뮬레이션이 완료되면, Results 디렉토리에 결과가 저장됨. 결과 중 Compact는 전체 layer에 대한 결과, Detail은 layer 별 결과, perPod은 첫 번째 (0, 0 위치) pod에 대한 결과를 나타냄.